Graphic Era deemed to be University DEHRADUN

PROJECT AND TEAM INFORMATION

Project Title

Local shop delivery system

Team Name:DSCPP-III-2026-T113 *Team #*	*Members*
Team member 1 (Team Lead)	*Bisht, Shaurya – 2510012399* *shauryabisht75@gmail.com* 
Team member 2	*Ranjan, Mayank – 2510011761* *mayankranjan56@gmail.com* 
Team member 3	*Vardhan, Raj – 2510010483* *adhikarirajvardhan@gmail.com* 
Team member 4	*Singh, Aradhay – 2510011780* *aradhay401@gmail.com* 

PHASE-III PROJECT PROGRESS DESCRIPTION

Project Abstract

The Local Shop Delivery System is a hyperlocal e-commerce platform implemented as a C++ console application.My project enables shops to sell products online through delivery agents.Blinkit and Zepto need their own stores and a big delivery network to work.Our project fills this gap by connecting shopkeepers,customers and delivery agents in one single platform.Shopkeepers manage(add,delete,view) their products and stock. Customers browse items, compare the price of the same item across nearby shops, place order and can track the order. Delivery agents are registered, get orders assigned to them and complete the deliveries in shortest possible time.In Phase-III, we all have finished the remaining modules.We build this project in C++ language and have used concept of object-oriented programming.With help of hash map customer finds products quickly, queue and linked list handle each order from start to finish, sorting is used to compare prices among items, and graph helps the delivery agent to pickup and deliver order.The data is stored in a MySQL database which is running on XAMPP.Payment can be done in cash on delivery, card,Upi.The final result is a tested system that takes order and deliver to the customer in shortest possible time.The project helps the shops to sell items online, gives customers the convenience of home delivery and gives delivery agents work to earn their living.

Updated Project Approach and Architecture

The system follows a five-layer architecture, implemented as a C++ console application.Splitting the system into layers keeps each module separate, so it is easier to test and maintain.
Input Layer: Menus based on the user's role, so the customer, shopkeeper and delivery agent each see their own options.
Processing Layer: Checks every input and asks the user to enter it again if it is wrong.
Core Logic Layer: A hash map finds the shops that sell a product quickly (constant time on average), and a FIFO queue holds the incoming orders. A linked list tracks each order's status: Pending, Accepted or Delivered. Sorting puts the lowest-priced shop first, and a graph of local areas with Dijkstra's algorithm finds the nearest delivery agent.
Database Layer: MySQL running on XAMPP stores users, products, orders and ratings.
Output Layer: Shows the results, order status and ratings on the console.
Object-Oriented Design: User is the base class. Shopkeeper, Customer and DeliveryAgent inherit from it.
I kept the layer names the same, so they still match your architecture diagram.**Order workflow:** The shopkeeper adds products. The customer compares prices and places an order, which goes into the queue as Pending. The nearest free agent is assigned and the order becomes Accepted. When the agent completes the delivery, the order is marked Delivered. After that, the customer rates the shop and the agent.
Technology stack: C++, MySQL, XAMPP, Visual Studio Code and Git/GitHub.
Communication and libraries: All three roles use the same console application, so no network protocol is needed between users. The program talks to the MySQL server through MySQL Connector/C++, sending SQL queries to the local XAMPP setup. The rest of the work is done with standard C++ libraries.
Design decisions: We chose Dijkstra's algorithm over BFS because the distances between areas are not equal. As planned in the scope, GPS and online payment are not included, and each agent handles one order at a time. Each member worked on their own Git branch, and the code was merged into the main branch only after the team lead reviewed it.

Tasks Completed (7 pts)

Task Completed	Team Member
1.Established the GitHub repository and folder structure; implemented the order queue and linked-list status flow; designed the MySQL tables (users, products, orders, ratings) in XAMPP; added input validation and error handling across all modules; integrated all branches into main and conducted final testing and bug fixing. 2.Developed the Product and Shop classes with hash-map lookup and stock management (add, update, delete, view); implemented the rating system for shops and delivery agents, with ratings stored in MySQL and average ratings shown to customers. 3.Developed the Customer class with cart, product browsing and order placement; implemented price comparison using sorting (lowest-priced shop first); built the order tracking screen displaying Pending, Accepted and Delivered status. 4.Developed the User base class and DeliveryAgent class using inheritance; implemented agent registration and assigned-order viewing; implemented nearest-agent assignment using Dijkstra's algorithm on the area graph and the delivery completion step (Accepted to Delivered). 5.Tested individual modules, resolved integration defects, and prepared the final report, presentation and live demonstration.	1.Shaurya Bisht (Team Lead) 2.Mayank Ranjan (Shop Management) 3.Raj Vardhan (Customer Module) 4.Aradhay Singh (Delivery Agent Module) 5.All members

Challenges/Roadblocks

1. Database connectivity: Linking MySQL Connector/C++ with XAMPP and VS Code gave us compilation and linker errors.
*Resolution: We first tested a small connection program on its own and followed the connector documentation step by step. Only then did we add it to the main code.
2. Nearest agent without GPS: Real-time location data was outside our scope.
*Resolution: We stored the local areas as a weighted graph with fixed distances between them. We used Dijkstra's algorithm because BFS only counts steps, not distances.
3. Invalid console input: The program crashed on negative prices, text instead of numbers and wrong menu choices.
*Resolution: We added input validation in every module, so the program asks the user to enter the value again instead of stopping.
4. Module integration: When we merged the four branches, the class interfaces did not match and we got merge conflicts.
*Resolution: We agreed on common class interfaces and kept our commits small. Merges into main were done only after the team lead reviewed them. 5. Data consistency: The order status in the queue and linked list could end up different from what was saved in MySQL.
*Resolution: We now update the database every time an order's status changes.
6. Agent unavailability: When every agent was busy, an order could not be assigned.
*Resolution: Such orders now wait in the queue as Pending until an agent is free.
7. Schedule constraints: The project ran at the same time as our exams and other subjects.
*Resolution: We broke the work into small tasks and checked progress in a short weekly team meeting.
Current limitations: The system runs on the console, uses a fixed area graph instead of GPS, and supports only cash on delivery. These are covered in the future scope.

Project Outcome/Deliverables

Our project is a fully working console-based e-commerce application called the Local Shop Delivery System.
The deliverables are:
1. **Shopkeeper module:** Lets shopkeepers manage their product catalogue and stock. They can add, update, delete and view products.
2. Customer module: Customers can browse products, compare the price of the same item across nearby shops (lowest price first), place orders and track them.
3. Delivery agent module: Agents can register, get orders assigned based on who is nearest, and mark deliveries as complete.
4. Rating system: Customers can rate both the shops and the delivery agents.
5. MySQL database: Tables for users, products, orders and ratings, along with the SQL script that creates them.
6. Source code: A GitHub repository with a README, a separate branch for each member and the full commit history.
7. Documentation: The final project report, the presentation and a live demonstration.
The project uses hash maps, queues, linked lists, sorting and a graph algorithm (Dijkstra's) from Data Structures. From OOP with C++, it uses classes, objects and inheritance.
The console version is a good base for a future mobile app and real-time delivery tracking.

Progress Overview

Completion status: At the end of Phase-II, the project stood at approximately 60 percent completion. All remaining planned work was completed in Phase-III, bringing the planned scope to 100 percent: nearest-agent assignment using Dijkstra's algorithm, delivery completion, order tracking, the rating system, input validation, module integration and final testing. All Phase-II test cases, including the previously pending agent-assignment test, now pass, along with the additional tests introduced in Phase-III.
Schedule analysis: The MySQL connectivity (Phase-II) and the nearest-agent logic required more time than planned, owing to connector set-up and additional testing of Dijkstra's algorithm on the area graph. Price comparison (Phase-II) and the rating system were completed ahead of schedule.
Outstanding items: None within the planned scope. The mobile application and real-time tracking were not part of the planned scope and are retained as future work.

Codebase Information

Repository: https://github.com/shauryabisht75-ui/Local-shop-delivery-system-
README: https://github.com/shauryabisht75-ui/Local-shop-delivery-system-/blob/main/README.md
Contributors: all four team members, added as collaborators.
Branch structure: main holds the integrated, final code. Each member developed on a separate branch, and all were merged into main after team lead review.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
1. Shop stock functions 2. Product lookup (hash map) 3. Customer order placing 4. Price comparison 5. Delivery agent assignment (Dijkstra) 6. Delivery completion 7. Rating system 8. Input validation 9.Input Validation 10.Integration (end-to-end)	1.Pass 2.Pass 3.Pass 4.Pass 5.Pass 6.Pass 7.Pass 8.Pass 9.Pass 10.Pass	1.Add, update, delete and view all worked correctly on the sample shops. 2.Products are found directly by ID or name, without going through the whole list. 3.The customer can browse products and place an order. The new order goes into the queue as Pending. 4.If the same product is sold in several shops, the cheapest shop is shown first (sorting). 5.This was pending in Phase-II and is now verified. The nearest free agent is assigned using the sample area graph. If all agents are busy, the order stays Pending. 6.Status changes from Pending → Accepted → Delivered and matches the MySQL record. 7.The agent completes the delivery and is free to take the next order. 8.Ratings are saved and the average rating shows correctly. 9.Negative prices, text instead of numbers and wrong menu choices ask the user to enter again, and the program does not crash. 10.The whole flow from shopkeeper → customer → delivery agent ran on sample data.

Deliverables Progress

1.Shopkeeper module—Completed—Catalogue and stock management
2.Customer module—Completed—Browsing, price comparison, order placement, order tracking
3.Order management—Completed—Queue and linked-list status flow; nearest-agent assignment
4.Delivery agent module—Completed—Registration, assigned orders, delivery completion
5.Rating system—Completed—Pending in Phase-II; now implemented
6.Data storage (MySQL)—Completed—Tables created; C++ connectivity verified
7.Final report—Completed—Submitted with this phase
8.Final presentation and demonstration —Completed—Prepared by all members
9.Mobile application and real-time tracking—Future scope—Outside the current project scope